

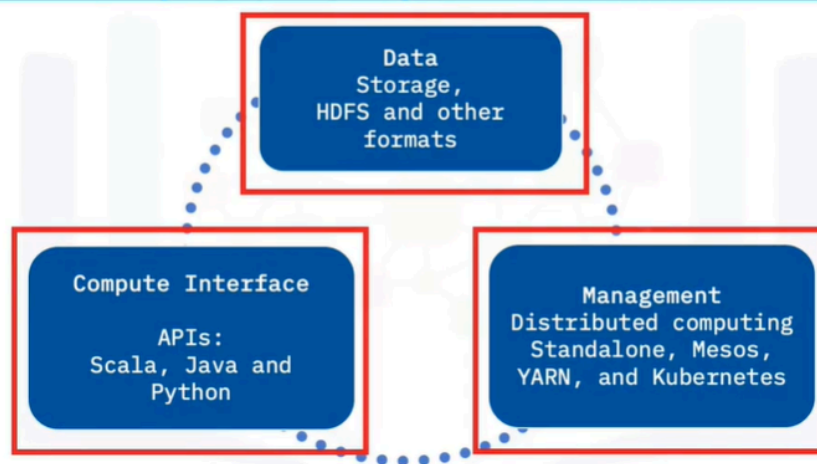
Scale out / Data Parallelism in Apache Spark

Sure! Let's dive into the concept of **Apache Spark Architecture** in simple terms.

Apache Spark is like a well-organized team working together to handle large amounts of data efficiently. Imagine a big office where different employees (called executors) are assigned tasks by their manager (the driver). The driver decides what needs to be done and distributes the work among the employees, who then complete their tasks and report back to the driver. This teamwork allows Spark to process big data quickly and effectively.

In Spark, there are three main components: data storage, high-level programming APIs, and a cluster management framework. The data storage is where all the information is kept, like a filing cabinet. The APIs are the tools that employees use to do their jobs, and the cluster management framework is like the office layout that helps everyone work together smoothly. By adding more employees (worker nodes), the office can handle even more tasks, making it easier to scale up when needed.

Apache Spark Components

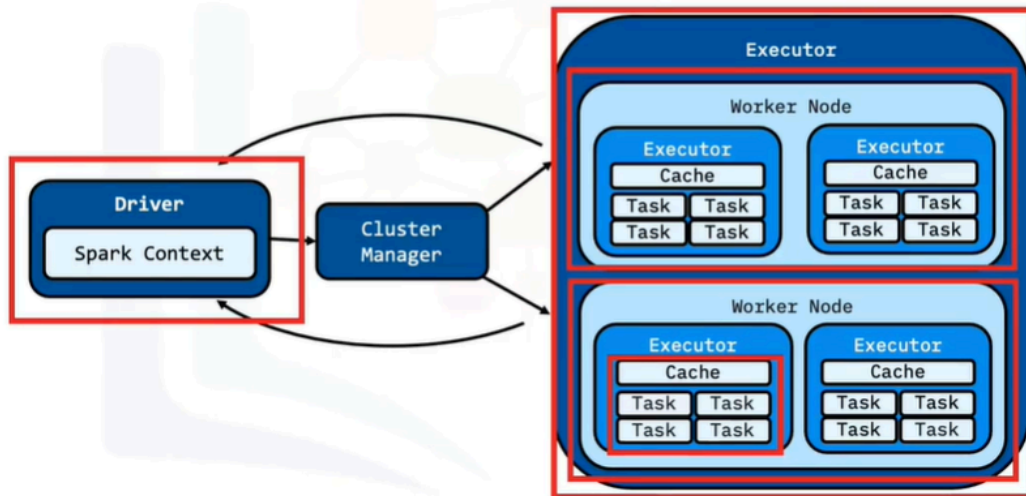


- Apache Spark architecture consists of three main components. The first component is data storage. Datasets load from data storage into memory. Any Hadoop compatible data source is acceptable. High-level programming APIs comprise the second component. Spark has APIs in Scala, Python, and Java.
- The final component is the cluster management framework, which handles the distributed computing aspects of Spark. Spark's cluster management framework can exist as a stand-alone server, Mesos or Yet Another Resource Network, or YARN. A cluster management framework is essential for scaling big data.
- Here is a simple visualization of the three pieces. The data from a Hadoop file system flows into the compute interface or API, which then flows into different nodes to perform distributed/parallel tasks.

Spark Core

- Is a base engine
 - Is fault-tolerant
 - Performs large scale parallel and distributed data processing
 - Manages memory
 - Schedules tasks
 - Houses APIs that define RDDs
 - Contains a distributed collection of elements that are parallelized across the cluster
-
- What is often referred to as "Spark" is the base engine formally called the "Spark Core." The fault-tolerant Spark Core is the base engine for large-scale parallel and distributed data processing. Spark Core manages memory and task scheduling. Spark Core also contains the APIs used to define RDDs and other datatypes. Spark Core also parallelizes a distributed collection of elements across the cluster.

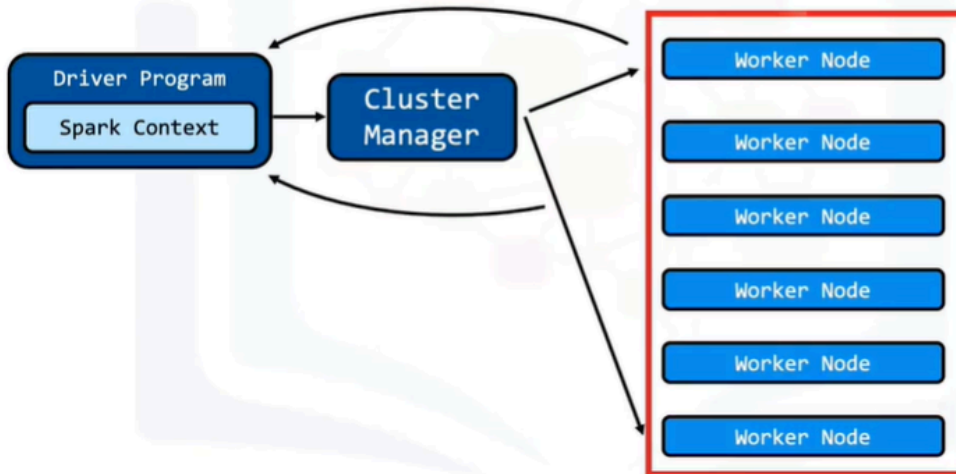
Scaling big data in Spark



- To understand how Spark scales with big data, let's review the Spark Application Architecture. The Spark Application consists of the driver program and the executor program.
- Executor programs run on worker nodes. Spark can start additional executor processes on a worker if there is enough memory and cores available. Similarly, executors can also take multiple cores for multithreaded calculations.
- Spark distributes RDDs among executors. Communication occurs among the driver and the executors. The driver contains the Spark jobs that the application needs to run and splits the jobs into tasks submitted to the executors.
- The driver receives the task results when the executors complete the tasks. If Apache Spark were a large organization or company, the driver code would be the executive management of that company that makes decisions about allocating work, obtaining capital, and more. The junior employees are the executors who do the jobs assigned to them with the resources provided.

- The worker nodes correspond to the physical office space that the employees occupy.

Scaling big data in Spark



- You can add additional worker nodes to scale big data processing incrementally.

Summary

In this video, you learned that:

- Apache Spark architecture consists of three main pieces; components data, compute input, and management
- The fault-tolerant Spark Core base engine performs large scale parallel and distributed data processing, manages memory, schedules tasks, and houses APIs that define RDDs
- The Spark driver program communicate with the cluster and then distributes RDDs among worker nodes.